工具函数

```cpp
#include <bits/stdc++.h>

const double EPS = 1e-8;

// 按 EPS 容差返回 -1、0、1
int sign(double value) {
    if (std::abs(value) < EPS) return 0;
    if (value > 0) return 1;
    return -1;
}

// 比较两数大小，返回 lhs - rhs 的符号
int cmp(double lhs, double rhs) {
    return sign(lhs - rhs);
}
```

点与线

```
struct Point {
    double x, y; // 既可表示点，也可表示向量

    Point(double xValue = 0, double yValue = 0)
        : x(xValue), y(yValue) {}

    friend Point operator+(Point a, Point b) {
        return Point(a.x + b.x, a.y + b.y);
    }

    friend Point operator-(Point a, Point b) {
        return Point(a.x - b.x, a.y - b.y);
    }

    friend Point operator/(Point a, double scale) {
        return Point(a.x / scale, a.y / scale);
    }

    friend Point operator*(Point a, double scale) {
        return Point(a.x * scale, a.y * scale);
    }

    friend Point operator*(double scale, Point b) {
        return Point(b.x * scale, b.y * scale);
    }

    // 向量点积；与标量乘法共用 *
    friend double operator*(Point a, Point b) {
        return a.x * b.x + a.y * b.y;
    }

    friend bool operator==(Point a, Point b) {
        return (!sign(a.x - b.x) && !sign(a.y - b.y));
    }

    friend bool operator!=(Point a, Point b) {
        return !(a == b);
    }

    // % 表示叉积：正值为逆时针；优先级高于 ^
```

```
    friend double operator%(Point a, Point b) {
        return a.x * b.y - a.y * b.x;
    }

    // 向量模长的平方与模长
    double lengthSquared() { return x * x + y * y; }

    double length() { return sqrt(x * x + y * y); }
} invalidPoint(1e20, 1e20); // 无有效结果时的哨兵

// 两点的欧氏距离
double distance(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

// 两点距离的平方，可避免开方
double distanceSquared(Point a, Point b) {
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
}

struct Segment {
    Point start, end; // 同时用于表示从 start 指向 end 的直线

    Segment() {}

    Segment(Point a, Point b) : start(a), end(b) {}

    friend double operator*(Segment a, Segment b) {
        return (a.end - a.start) * (b.end - b.start);
    }

    friend double operator%(Segment a, Segment b) {
        return (a.end - a.start) % (b.end - b.start);
    }

    friend double operator%(Segment a, Point b) {
        return (a.end - a.start) % (b - a.start);
    }

    friend double operator%(Point a, Segment b) {
        return (a - b.start) % (b.end - b.start);
    }
```

```
    // 线段相等不区分端点顺序
    friend bool operator==(Segment a, Segment b) {
        return (a.start == b.start && a.end == b.end) ||
               (a.start == b.end && a.end == b.start);
    }

    void reverse() {
        std::swap(start, end);
    }

    double length() { return distance(start, end); }
};

// 点到所在直线的距离；两端点须不同
double pointToLineDistance(Segment line, Point point) {
    return std::abs((line.end - line.start) % (point - line.start)) /
        line.length();
}

// 点到线段的距离；线段须非退化
double pointToSegmentDistance(Segment segment, Point point) {
    int startDot = sign((point - segment.start) *
                        (segment.end - segment.start));
    int endDot = sign((point - segment.end) * (segment.start - segment.end));
    // 垂足在线段内取垂距，否则取较近端点
    if (startDot >= 0 && endDot >= 0) {
        return pointToLineDistance(segment, point);
    }
    return std::min(distance(segment.start, point),
                    distance(segment.end, point));
}

// 直线上的垂足，可能落在线段外；两端点须不同
Point projectToLine(Segment line, Point point) {
    auto dir = line.end - line.start;
    return line.start + dir * ((point - line.start) * dir) /
                        (dir.x * dir.x + dir.y * dir.y);
}

// 两条所在直线的交点；要求不平行
Point lineIntersection(Segment a, Segment b) {
    auto crossStart = (a.end - a.start) % (b.start - a.start);
    auto crossEnd = (a.end - a.start) % (b.end - a.start);
```

```
    auto x = (crossEnd * b.start.x - crossStart * b.end.x) /
             (crossEnd - crossStart);
    auto y = (crossEnd * b.start.y - crossStart * b.end.y) /
             (crossEnd - crossStart);
    return Point(x, y);
}

// 共线且夹在两端点之间，包括端点
bool isPointOnSegment(Segment segment, Point point) {
    Point toStart = segment.start - point, toEnd = segment.end - point;
    if (sign(toStart % toEnd) == 0 && sign(toStart * toEnd) <= 0) return true;
    return false;
}

// 线段相交判定，包括端点接触及共线重叠
bool segmentsIntersect(Segment a, Segment b) {
    // 两线段须互相跨立
    double c1 = (a.end - a.start) % (b.start - a.start),
           c2 = (a.end - a.start) % (b.end - a.start);
    double c3 = (b.end - b.start) % (a.start - b.start),
           c4 = (b.end - b.start) % (a.end - b.start);

    // 有端点共线时，单独检查是否接触或重叠
    if (!sign(c1) || !sign(c2) || !sign(c3) || !sign(c4)) {
        bool startBOnA = isPointOnSegment(a, b.start);
        bool endBOnA = isPointOnSegment(a, b.end);
        bool startAOnB = isPointOnSegment(b, a.start);
        bool endAOnB = isPointOnSegment(b, a.end);
        bool touches = (startBOnA | endBOnA | startAOnB | endAOnB);
        return touches;
    }
    return (sign(c1) * sign(c2) < 0 && sign(c3) * sign(c4) < 0);
}

// 最近点对距离；处理非空闭区间 [left, right]
double closestPairDistance(int left, int right, Point points[], int strip[]) {
    /*
    调用前按 x 排序；strip 至少能容纳 n + 1 个下标
    std::sort(points, points + n, [&](const Point &p1, const Point &p2) {
        return p1.x < p2.x;
    });
    */
    if (right - left == 0)
```

```
        return std::numeric_limits<double>::infinity();
    if (right - left == 1)  // 只有两点时直接计算
        return distance(points[left], points[right]);
    int mid = (left + right) >> 1;
    double best = std::min(closestPairDistance(left, mid, points, strip),
                           closestPairDistance(mid + 1, right, points, strip));
    int count = 0;
    for (int i = left; i <= right; i++)
        // 跨分界线的更优点对，只会落在宽度 2 * best 的条带内
        if (points[i].x >= points[mid].x - best && points[i].x <=
            points[mid].x + best)
            strip[++count] = i;
    std::sort(strip + 1, strip + count + 1, [&](int i, int j) {
        return points[i].y < points[j].y;
    }); // 按 y 排序，便于提前终止扫描
    for (int i = 1; i <= count; i++)
        for (int j = i + 1; j <= count; j++) {
            if (points[strip[j]].y >= points[strip[i]].y + best) break;
            // 纵向距离已不小于 best 时，后续点无需再比较
            best = std::min(best, distance(points[strip[i]], points[strip[j]]));
        }
    return best;
}
```

圆

```cpp
struct Circle {
    Point center;
    double radius;

    Circle(Point centerValue, double radiusValue)
        : center(centerValue), radius(radiusValue) {}

    Circle(double x = 0, double y = 0, double radiusValue = 0) {
        center.x = x, center.y = y, radius = radiusValue;
    }

    // 极角单位为弧度，从 x 轴正向逆时针计量
    Point pointAtAngle(double angle) {
        double offsetX = cos(angle) * radius;
        double offsetY = sin(angle) * radius;
        if (sign(offsetX) == 0) offsetX = 0.0;
        if (sign(offsetY) == 0) offsetY = 0.0;
        return Point(center.x + offsetX, center.y + offsetY);
    }

    // 沿圆心指向 p 的射线取圆上点；p 不能为圆心
    Point pointToward(Point p) {
        auto dir = p - center;
        return center + dir * radius / dir.length();
    }
};

// 圆的位置关系；要求半径为正且两圆不重合
int circleRelation(Circle a, Circle b) {
    // 0 内含，1 内切，2 相交，3 外切，4 外离
    int relation = cmp(distance(a.center, b.center), a.radius + b.radius);
    if (relation > 0) return 4; // 外离
    if (relation == 0) return 3; // 外切
    relation = cmp(distance(a.center, b.center),
                   std::abs(a.radius - b.radius));
    if (relation > 0) return 2; // 相交
    if (relation == 0) return 1; // 内切
    return 0; // 内含
}
```

```cpp
// 两圆交点；要求有交点、半径为正且圆心不同
std::array<Point, 2> circleIntersections(Circle c1, Circle c2) {
    Point centerDir = c2.center - c1.center; // 从 c1 圆心指向 c2 圆心
    double centerDist = centerDir.length(); // 圆心距
    // 余弦定理求圆心连线与交点方向的夹角
    double offsetAngle = acos((c1.radius * c1.radius + centerDist * centerDist -
        c2.radius * c2.radius) / (2.0 * c1.radius * centerDist));
    double baseAngle = atan2(centerDir.y, centerDir.x); // 圆心连线的极角
    return {c1.pointAtAngle(baseAngle + offsetAngle),
        c1.pointAtAngle(baseAngle - offsetAngle)};
}

// 三角形内切圆；要求三点不共线
Circle triangleIncircle(Point p1, Point p2, Point p3) {
    Circle c;
    auto side12 = distance(p1, p2), side13 = distance(p1, p3),
         side23 = distance(p2, p3);
    // 内心按对边长加权；半径为两倍面积除以周长
    c.radius = std::abs((p2.x - p1.x) * (p3.y - p1.y) -
            (p2.y - p1.y) * (p3.x - p1.x)) / (side12 + side13 + side23);
    c.center.x = (side23 * p1.x + side12 * p3.x + side13 * p2.x) / (side12 +
        side13 + side23);
    c.center.y = (side23 * p1.y + side12 * p3.y + side13 * p2.y) / (side12 +
        side13 + side23);
    return c;
}

// 三角形外接圆；要求三点不共线
Circle triangleCircumcircle(Point p1, Point p2, Point p3) {
    Circle c;
    c.radius = distance(p1, p2) * distance(p2, p3) * distance(p1, p3) /
        std::abs((p2 - p1) % (p3 - p1)) / 2.0;
    // 联立两条垂直平分线，用行列式求圆心
    auto a11 = (p2.x - p1.x);
    auto a12 = (p2.y - p1.y);
    auto b1 = (p1.x * (p2.x - p1.x) + p1.y * (p2.y - p1.y) + 0.5 *
        distanceSquared(p1, p2));
    auto a21 = (p3.x - p1.x);
    auto a22 = (p3.y - p1.y);
    auto b2 = (p1.x * (p3.x - p1.x) + p1.y * (p3.y - p1.y) + 0.5 *
        distanceSquared(p1, p3));
    auto det = (a11 * a22 - a21 * a12);
    c.center.x = (b1 * a22 - b2 * a12) / det;
```

```
    c.center.y = (a11 * b2 - b1 * a21) / det;
    return c;
}

// 所在直线与圆的交点数；直线两端点须不同
int lineCircleCount(Segment line, Circle circle) {
    int relation = cmp(pointToLineDistance(line, circle.center),
        circle.radius);
    if (relation == 0) return 1; // 相切
    if (relation == 1) return 0; // 不相交
    return 2;
}

// 直线与圆交点：无交点返回哨兵，相切返回重复点
std::array<Point, 2> lineCircleIntersections(Circle circle, Segment line) {
    auto foot = projectToLine(line, circle.center);
    int count = lineCircleCount(line, circle);
    if (count == 0) return {invalidPoint, invalidPoint};
    if (count == 1) return {foot, foot};
    // 垂足为弦中点，offset 为半弦长
    double offset = sqrt(circle.radius * circle.radius -
                         (foot - circle.center).lengthSquared());
    auto dir = line.end - line.start;
    Point unitDir = dir / dir.length();
    return {foot + unitDir * offset, foot - unitDir * offset};
}

// 点到圆的切点：圆内返回哨兵，圆上返回重复点
std::array<Point, 2> pointCircleTangents(Circle circle, Point p) {
    auto centerDist = distance(p, circle.center);
    int relation = cmp(centerDist, circle.radius);
    if (relation < 0) return {invalidPoint, invalidPoint};  // 点在圆内
    else if (relation == 0) return {p, p}; // 圆上返回重复切点

    // 点在圆外，有两个切点
    double baseAngle = atan2(p.y - circle.center.y, p.x - circle.center.x);
    double offsetAngle = acos(circle.radius / centerDist);
    return {circle.pointAtAngle(baseAngle - offsetAngle),
        circle.pointAtAngle(baseAngle + offsetAngle)};
}

// 公切线在 c1 上的切点；要求 c1.radius >= c2.radius
std::vector<Point> circleTangentPoints(Circle c1, Circle c2) {
```

```
    int relation = circleRelation(c1, c2);
    std::vector<Point> tangentPoints;
    auto centerDist = distance(c1.center, c2.center);
    if (relation == 0) return tangentPoints;
    double baseAngle = atan2(c2.center.y - c1.center.y,
                             c2.center.x - c1.center.x);
    // baseAngle 为 c1 指向 c2 的极角；两圆须不重合
    if (relation == 1) { // 内切
        tangentPoints.push_back(c1.pointAtAngle(baseAngle));
        return tangentPoints;
    }
    // 两条外公切线
    double outerAngle = acos((c1.radius - c2.radius) / centerDist);
    tangentPoints.push_back(c1.pointAtAngle(baseAngle + outerAngle));
    tangentPoints.push_back(c1.pointAtAngle(baseAngle - outerAngle));
    // 外切时只有一条内公切线
    if (relation == 3)
        tangentPoints.push_back(c1.pointAtAngle(baseAngle));
    else if (relation == 4) { // 外离时有两条内公切线
        double innerAngle = acos((c1.radius + c2.radius) / centerDist);
        tangentPoints.push_back(c1.pointAtAngle(baseAngle + innerAngle));
        tangentPoints.push_back(c1.pointAtAngle(baseAngle - innerAngle));
    }
    return tangentPoints;
}

// 随机增量法求最小覆盖圆；n >= 1，会打乱输入
Circle minimumEnclosingCircle(Point points[], int n) {
    std::random_device seed;
    std::mt19937 rng(seed());
    std::shuffle(points, points + n, rng);
    Circle c(points[0], 0);
    for (int i = 1; i < n; i++) {
        if (cmp(distance(c.center, points[i]), c.radius) > 0) {
            // 新点在圆外，依次固定一、二、三个边界点
            c.radius = 0;
            c.center = points[i];
            for (int j = 0; j < i; j++) {
                if (cmp(distance(c.center, points[j]), c.radius) > 0) {
                    c.radius = distance(points[j], points[i]) / 2;
                    c.center = {(points[j].x + points[i].x) / 2, (points[j].y +
                        points[i].y) / 2};
                    for (int k = 0; k < j; k++) {
```

```
                        if (cmp(distance(c.center, points[k]),
                            c.radius) > 0) {
                            c = triangleCircumcircle(points[i], points[j],
                                points[k]);
                        }
                    }
                }
            }
        }
    }
    return c;
}
```

多边形

```
// 简单多边形重心，凹凸均可；顶点有序且面积非零
Point polygonCentroid(Point points[], int n) {
    double areaSum = 0.0, weightedX = 0, weightedY = 0;
    Point p1 = points[0], p2 = points[1], p3;
    for (int i = 2; i <= n - 1; i++) {
        p3 = points[i];
        // 按有向三角形面积加权，凹多边形也适用
        double area = (p2 - p1) % (p3 - p2) / 2.0;
        areaSum += area;
        weightedX += (p1.x + p2.x + p3.x) * area;
        weightedY += (p1.y + p2.y + p3.y) * area;
        p2 = p3;
    }
    return Point(weightedX / (3.0 * areaSum), weightedY / (3.0 * areaSum));
}

// 点与简单多边形的位置：0 外部，1 边界，2 内部
int pointInPolygon(int n, Point points[], Point query) {
    int crossings = 0;
    Point start, end;
    for (int i = 0; i < n; i++) {
        start = points[i];
        end = points[(i + 1) % n];
        // 边界优先判定
        if (isPointOnSegment(Segment(start, end), query)) return 1;
        // 半开区间计数，避免射线经过顶点时重复
        if ((sign(start.y - query.y) > 0 != sign(end.y - query.y) > 0) &&
            sign(query.x - (query.y - start.y) * (start.x - end.x) / (start.y -
                end.y) - start.x) < 0)
            crossings++;
    }
    if (crossings & 1) return 2; // 右向射线穿边次数为奇数
    else return 0; // 偶数次则在外部
}

// 鞋带公式求简单多边形面积，顺逆时针均可
double polygonArea(Point points[], int n) {
    double area = 0;
    for (int i = 0; i < n; i++)
        area += points[i] % points[(i + 1) % n];
```

```
    return std::abs(area) / 2.0;
}

// 凸多边形在有向直线左侧的面积；直线须非退化
double cutPolygonArea(Point points[], int n, Segment line) {
    Point dir = line.end - line.start;
    // 顶点按逆时针排列，动态保存裁剪结果
    std::vector<Point> clipped;
    for (int i = 0; i < n; i++) {
        int sideStart = sign((points[i] - line.start) % dir);
        int sideEnd = sign((points[(i + 1) % n] - line.start) % dir);
        if (sideStart <= 0) clipped.push_back(points[i]);
        // 当前叉积顺序下 <= 0 为左侧；右侧改为 >= 0
        if (sideStart * sideEnd == -1) {
            clipped.push_back(lineIntersection(line, Segment(points[i],
                points[(i + 1) % n])));
        }
    }
    return polygonArea(clipped.data(), static_cast<int>(clipped.size()));
}

// 读入 n 个整点，输出最大曼哈顿距离
// 曼哈顿距离：abs(x1 - x2) + abs(y1 - y2)
// 切比雪夫距离：max(abs(x1 - x2), abs(y1 - y2))
// 变换后，两坐标的极差最大值即原坐标的最大曼哈顿距离
// 正变换：(x, y) -> (x + y, x - y)
// 逆变换：(x, y) -> ((x + y) / 2, (x - y) / 2)
// n >= 1；变换值及最终距离须能存入 long long
void solveMaxManhattanDistance() {
    int n;
    std::cin >> n;
    long long a, b;
    std::cin >> a >> b;
    long long minX = a + b, maxX = a + b;
    long long minY = a - b, maxY = a - b;
    for (int i = 1; i < n; i++) {
        std::cin >> a >> b;
        long long x = a + b, y = a - b;
        maxX = std::max(x, maxX), maxY = std::max(y, maxY);
        minX = std::min(x, minX), minY = std::min(y, minY);
    }
    std::cout << std::max(maxX - minX, maxY - minY);
}
```

```cpp
// Pick 定理：单位正方形格点中，A = I + B / 2 - 1
// 顶点须为整点并按边界有序，组成非退化简单多边形
struct IntPoint {
    long long x, y;
};

// 返回 {I, B}，须能存入 long long；中间值须在 __int128 范围内
std::array<long long, 2> countLatticePoints(int n, IntPoint points[]) {
    auto gcd = [](__int128 a, __int128 b) {
        while (b != 0) {
            __int128 remainder = a % b;
            a = b, b = remainder;
        }
        return a;
    };
    __int128 twiceArea = 0, boundaryCount = 0;
    for (int i = 0; i < n; i++) {
        IntPoint a = points[i], b = points[(i + 1) % n];
        // 先提升类型再做差或相乘，避免在 long long 中溢出
        __int128 dx = static_cast<__int128>(b.x) - a.x;
        __int128 dy = static_cast<__int128>(b.y) - a.y;
        // 每条边计入一个端点，避免公共顶点重复
        boundaryCount += gcd(dx < 0 ? -dx : dx, dy < 0 ? -dy : dy);
        twiceArea += static_cast<__int128>(a.x) * b.y -
                     static_cast<__int128>(a.y) * b.x;
    }
    if (twiceArea < 0) twiceArea = -twiceArea;
    return {static_cast<long long>((twiceArea - boundaryCount) / 2 + 1),
            static_cast<long long>(boundaryCount)};
}

// Andrew 凸包，返回顶点数；输入点会被排序
int convexHull(int n, Point points[], Point hull[]) {
    // n >= 2，要求点互异；hull 容量至少为 2 * n
    // 输入、输出下标均从 0 开始
    std::sort(points, points + n, [&](const Point &a, const Point &b) {
        // 排序使用精确字典序，EPS 比较不满足严格弱序
        if (a.x == b.x) return a.y < b.y;
        return a.x < b.x;
    });
    int size = 0;
    for (int i = 0; i < n; i++) {
```

```
        while (size > 1 && (hull[size - 1] - hull[size - 2]) % (points[i] -
            hull[size - 2]) <= 0) --size;
        // 两处 <= 均改为 < 可保留边上点；全共线需另行处理
        // 弹出非左转点，结果为逆时针
        hull[size++] = points[i];
    }
    int lowerSize = size;
    for (int i = n - 2; i >= 0; i--) {
        while (size > lowerSize && (hull[size - 1] - hull[size - 2]) %
            (points[i] - hull[size - 2]) <= 0) --size;
        hull[size++] = points[i];
    }
    --size;
    // 删除重复的起点，输出不重复首尾
    return size;
}

// c 到 a、b 所在直线的距离；要求 a != b
double pointToSupportingLineDistance(Point a, Point b, Point c) {
    return std::abs((b - a) % (c - a)) / distance(a, b);
}

// 旋转卡壳求凸包直径，即最远点对距离
double convexDiameter(int n, Point hull[]) {
    // n >= 2；顶点沿凸包有序，n >= 3 时要求严格凸
    if (n == 2) return distance(hull[1], hull[0]);
    int opposite = 0;
    double diameter = 0;
    for (int i = 0; i < n; i++) {
        while (pointToSupportingLineDistance(hull[i], hull[(i + 1) % n],
            hull[opposite]) <=
               pointToSupportingLineDistance(hull[i], hull[(i + 1) % n],
                   hull[(opposite + 1) % n]))
            opposite = (opposite + 1) % n;
        diameter = std::max(diameter, distance(hull[i], hull[opposite]));
        diameter = std::max(diameter, distance(hull[(i + 1) % n],
            hull[opposite]));
    }
    return diameter;
}

// 检查有序多边形是否为逆时针凸多边形
bool isConvexPolygon(int n, Point points[]) {
```

```
    // n >= 3；前提是输入为简单多边形
    // 顺时针输入时，将 > 改成 <
    // 当前允许相邻边共线；严格凸时将 > 改成 >=
    for (int i = 0; i < n; i++) {
        if ((points[i] - points[(i + 1) % n]) % (points[(i + 2) % n] -
            points[(i + 1) % n]) > 0)
            return false;
    }
    return true;
}

// 半平面交：取各有向直线左侧，要求交集有界且非退化
std::vector<Point> halfPlaneIntersection(std::vector<Segment> lines) {
    auto angle = [](const Segment &line) {
        auto dir = line.end - line.start;
        return atan2(dir.y, dir.x);
    };
    // 排序不使用 EPS；端点坐标须有限且两端点不同
    std::sort(lines.begin(), lines.end(), [&](const Segment &a,
        const Segment &b) {
        return angle(a) < angle(b);
    });
    // 以每组首条直线的极角为基准，仅保留更紧的同向约束
    std::vector<Segment> uniqueLines;
    for (std::size_t i = 0; i < lines.size();) {
        double baseAngle = angle(lines[i]);
        Point dir = lines[i].end - lines[i].start;
        Segment best = lines[i];
        std::size_t j = i + 1;
        while (j < lines.size() && cmp(angle(lines[j]), baseAngle) == 0) {
            if (sign(dir % (lines[j].start - best.start)) > 0)
                best = lines[j];
            ++j;
        }
        uniqueLines.push_back(best);
        i = j;
    }
    lines.swap(uniqueLines);
    if (lines.size() < 3) return {};
    // b、c 的交点是否落在 a 的右侧
    auto isOutside = [&](const Segment &a, const Segment &b, const Segment &c) {
        auto intersection = lineIntersection(b, c);
        return sign((intersection - a.end) % (a.start - a.end)) < 0;
```

```
    };
    // 同向去重后，用双端队列维护有效边界
    std::deque active{lines[0], lines[1]};
    for (int i = 2; i < lines.size(); i++) {
        while (active.size() > 1 &&
               isOutside(lines[i], active.end()[-1], active.end()[-2]))
            active.pop_back();
        while (active.size() > 1 &&
               isOutside(lines[i], active.begin()[0], active.begin()[1]))
            active.pop_front();
        active.push_back(lines[i]);
    }
    while (active.size() > 1 &&
           isOutside(active.begin()[0], active.end()[-2], active.end()[-1]))
        active.pop_back();
    while (active.size() > 1 &&
           isOutside(active.end()[-1], active.begin()[0], active.begin()[1]))
        active.pop_front();
    std::vector<Point> polygon;
    for (int i = 0; i + 1 < active.size(); i++)
        polygon.push_back(lineIntersection(active[i], active[i + 1]));
    if (active.size() > 2) // 队列至少三条边时才闭合
        polygon.push_back(lineIntersection(active.front(), active.back()));
    return polygon;
}
```